

An introduction to Numpy

Overview

NumPy and SciPy are open-source add-on modules to Python that provide common mathematical and numerical routines in pre-compiled, fast functions. These are highly mature packages that provide numerical functionality that meets, or perhaps exceeds, that associated with commercial software like MatLab. The NumPy (Numeric Python) package provides basic routines for manipulating large arrays and matrices of numeric data. The SciPy (Scientific Python) package extends the functionality of NumPy with a substantial collection of useful algorithms like minimization, Fourier transformation, regression, and other applied mathematical techniques.

Installation

If you installed the Anaconda distribution, then you should be ready to go. If not, then you will have to install these add-ons manually after installing Python, in the order of NumPy and then SciPy. Installation files are available at:

<https://numpy.org>

<https://scipy.org/>

Other resources

The NumPy and SciPy development community maintains an extensive online documentation system, including user guides and tutorials, at:

<https://docs.scipy.org/doc/>

Importing the NumPy module

There are several ways to import NumPy. The standard approach is to use a simple import statement:

```
>>> import numpy
```

However, for large amounts of calls to NumPy functions, it can become tedious to write `numpy.X` over and over again. Instead, it is common to import under the briefer name `np`:

```
>>> import numpy as np
```

This statement will allow us to access NumPy objects using `np.X` instead of `numpy.X`. It is also possible to import NumPy directly into the current namespace so that we don't have to use dot notation at all, but rather simply call the functions as if they were built-in:

```
>>> from numpy import *
```

However, this strategy is usually frowned upon in Python programming because it starts to remove some of the nice organization that modules provide. For the remainder of this tutorial, we will assume that the `import numpy as np` has been used.

Arrays

The central feature of NumPy is the *array* object class. Arrays are similar to lists in Python, except that every element of an array must be of the same type, typically a numeric type like `float` or `int`. Arrays make operations with large amounts of numeric data very fast and are generally much more efficient than lists.

An array can be created from a list:

```
>>> a = np.array([1, 4, 5, 8], float)
>>> a
array([ 1.,  4.,  5.,  8.])
>>> type(a)
<class 'numpy.ndarray'>
```

Here, the function `array` takes two arguments: the list to be converted into the array and the type of each member of the list. Array elements are accessed, sliced, and manipulated just like lists:

```
>>> a[:2]
array([ 1.,  4.])
>>> a[3]
8.0
>>> a[0] = 5. >>> a
array([ 5.,  4.,  5.,  8.])
```

Arrays can be multidimensional. Unlike lists, different axes are accessed using commas inside bracket notation. Here is an example with a two-dimensional array (e.g., a matrix):

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
>>> a
array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.]])
>>> a[0,0]
1.0
>>> a[0,1]
2.0
```

Array slicing works with multiple dimensions in the same way as usual, applying each slice specification as a filter to a specified dimension. Use of a single ":" in a dimension indicates the use of everything along that dimension:

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
```

```
>>> a[1,:]
array([ 4.,  5.,  6.])
>>> a[:,2] array([
3.,  6.]) >>> a[-
1:,-2:] array([[
5.,  6.]])
```

The `shape` property of an array returns a tuple with the size of each array dimension:

```
>>> a.shape
(2, 3)
```

The `dtype` property tells you what type of values are stored by the array:

```
>>> a.dtype
dtype('float64')
```

Here, `float64` is a numeric type that NumPy uses to store double-precision (8-byte) real numbers, similar to the `float` type in Python.

When used with an array, the `len` function returns the length of the first axis:

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
>>> len(a)
2
```

The `in` statement can be used to test if values are present in an array:

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
>>> 2 in a
True
>>> 0 in a
False
```

Arrays can be reshaped using tuples that specify new dimensions. In the following example, we turn a ten-element one-dimensional array into a two-dimensional one whose first axis has five elements and whose second axis has two elements:

```
>>> a = np.array(range(10), float)
>>> a
array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.])
>>> a = a.reshape((5, 2))
>>> a
array([[ 0.,  1.],
       [ 2.,  3.],
       [ 4.,  5.],
       [ 6.,  7.],
       [ 8.,  9.]])
>>> a.shape
(5, 2)
```

Notice that the `reshape` function creates a new array and does not itself modify the original array.

Keep in mind that Python's name-binding approach still applies to arrays. The `copy` function can be used to create a separate copy of an array in memory if needed:

```
>>> a = np.array([1, 2, 3], float)
>>> b = a
>>> c = a.copy()
>>> a[0] = 0
>>> a
array([0., 2., 3.])
>>> b
array([0., 2., 3.])
>>> c
array([1., 2., 3.])
```

Lists can also be created from arrays:

```
>>> a = np.array([1, 2, 3], float)
>>> a.tolist()
[1.0, 2.0, 3.0]
>>> list(a)
[1.0, 2.0, 3.0]
```

One can convert the raw data in an array to a binary string (i.e., not in human-readable form) using the `tostring` function. The `fromstring` function then allows an array to be created from this data later on. These routines are sometimes convenient for saving large amount of array data in binary files that can be read later on:

```
>>> a = array([1, 2, 3], float)
>>> s = a.tostring()
>>> s
'\x00\x00\x00\x00\x00\x00\xf0?\x00\x00\x00\x00\x00\x00\x00@\x00\x00\x00\x00\x00\x00\x08@' >>>
np.fromstring(s)
array([ 1.,  2.,
        3.])
```

One can fill an array with a single value:

```
>>> a = array([1, 2, 3], float)
>>> a
array([ 1.,  2.,  3.])
>>> a.fill(0) >>> a
array([ 0.,  0.,  0.])
```

Transposed versions of arrays can also be generated, which will create a new array with the final two axes switched:

```
>>> a = np.array(range(6), float).reshape((2, 3))
>>> a
array([[ 0.,  1.,  2.],
       [ 3.,  4.,  5.]]) >>> a.transpose()
array([[ 0.,  3.],
       [ 1.,  4.],
       [ 2.,  5.]])
```

One-dimensional versions of multi-dimensional arrays can be generated with `flatten`:

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
>>> a
array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.]])
>>> a.flatten()
array([ 1.,  2.,  3.,  4.,  5.,  6.]])
```

Two or more arrays can be concatenated together using the `concatenate` function with a tuple of the arrays to be joined:

```
>>> a = np.array([1,2], float)
>>> b = np.array([3,4,5,6], float)
>>> c = np.array([7,8,9], float)
>>> np.concatenate((a, b, c)) array([1., 2., 3., 4., 5., 6., 7., 8., 9.])
```

If an array has more than one dimension, it is possible to specify the axis along which multiple arrays are concatenated. By default (without specifying the axis), NumPy concatenates along the first dimension:

```
>>> a = np.array([[1, 2], [3, 4]], float)
>>> b = np.array([[5, 6], [7,8]], float)
>>>
>>> np.concatenate((a,b))
array([[ 1.,  2.],
       [ 3.,  4.],
       [ 5.,  6.],
       [ 7.,  8.]])
>>> np.concatenate((a,b),
axis=0) array([[ 1.,  2.],
               [ 3.,  4.],
               [ 5.,  6.],
               [ 7.,  8.]])
>>> np.concatenate((a,b),
axis=1) array([[ 1.,  2.,  5.,
                 6.],
               [ 3.,  4.,  7.,
                 8.]])
```

Finally, the dimensionality of an array can be increased using the `newaxis` constant in bracket notation:

```
>>> a = np.array([1, 2, 3], float)
>>> a
array([1., 2., 3.])
>>> a[:,np.newaxis]
array([[ 1.],
       [ 2.],
       [ 3.]])
>>> a[:,np.newaxis].shape
(3,1)
>>> b[np.newaxis,:] array([[ 1.,  2.,  3.]]) >>> b[np.newaxis,:].shape
(1,3)
```

Notice here that in each case the new array has two dimensions; the one created by `newaxis` has a length of one. The `newaxis` approach is convenient for generating the properdimensioned arrays for vector and matrix mathematics.

Other ways to create arrays

The `arange` function is similar to the `range` function but returns an array instead of a list:

```
>>> np.arange(5, dtype=float)
array([ 0.,  1.,  2.,  3.,  4.]) >>> np.arange(1, 6, 2,
dtype=int) array([1, 3, 5])
```

The functions `zeros` and `ones` create new arrays of specified dimensions filled with these values. These are perhaps the most commonly used functions to create new arrays:

```
>>> np.ones((2,3),
dtype=float) array([[ 1.,  1.,  1.],
[ 1.,  1.,  1.]])
>>> np.zeros(7, dtype=int)
array([0, 0, 0, 0, 0, 0, 0])
```

The `zeros_like` and `ones_like` functions create a new array with the same dimensions and type of an existing one:

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
>>> np.zeros_like(a)
array([[ 0.,  0.,  0.],
[ 0.,  0.,  0.]]) >>>
np.ones_like(a)
array([[ 1.,  1.,  1.],
[ 1.,  1.,  1.]])
```

There are also a number of functions for creating special matrices (2D arrays). To create an identity matrix of a given size,

```
>>> np.identity(4,
dtype=float) array([[ 1.,  0.,  0.,  0.],
[ 0.,  1.,  0.,  0.],
[ 0.,  0.,  1.,  0.],
[ 0.,  0.,  0.,  1.]])
```

The `eye` function returns matrices with ones along the *k*th diagonal:

```
>>> np.eye(4, k=1, dtype=float) array([[ 0.,  1.,  0.,  0.],
[ 0.,  0.,  1.,  0.],
[ 0.,  0.,  0.,  1.],
[ 0.,  0.,  0.,  0.]])
```

Array mathematics

When standard mathematical operations are used with arrays, they are applied on an element-by-element basis. This means that the arrays should be the same size during addition, subtraction, etc.:

```
>>> a = np.array([1,2,3], float)
>>> b = np.array([5,2,6], float)
>>> a + b array([6., 4., 9.])
>>> a - b
array([-4.,  0., -3.])
>>> a * b
array([5., 4., 18.])
```

```
>>> b / a array([5., 1., 2.])
>>> a % b array([1., 0., 3.])
>>> b**a array([5., 4., 216.])
```

For two-dimensional arrays, multiplication remains elementwise and does *not* correspond to matrix multiplication.

```
>>> a = np.array([[1,2], [3,4]], float)
>>> b = np.array([[2,0], [1,3]], float)
>>> a * b array([[2., 0.], [3., 12.]])
```

Errors are thrown if arrays do not match in size:

```
>>> a = np.array([1,2,3], float)
>>> b = np.array([4,5], float)
>>> a + b
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: shape mismatch: objects cannot be broadcast to a single shape
```

However, arrays that do not match in the number of dimensions will be *broadcasted* by Python to perform mathematical operations. This often means that the smaller array will be repeated as necessary to perform the operation indicated. Consider the following:

```
>>> a = np.array([[1, 2], [3, 4], [5, 6]], float)
>>> b = np.array([-1, 3], float)
>>> a
array([[ 1.,  2.],
       [ 3.,  4.],
       [ 5.,  6.]])
>>> b
array([-1.,  3.])
>>> a + b
array([[ 0.,  2.],
       [ 5.,  7.],
       [ 4.,  9.]])
```

Here, the one-dimensional array `b` was broadcasted to a two-dimensional array that matched the size of `a`. In essence, `b` was repeated for each item in `a`, as if it were given by

```
array([[ -1.,  3.],
       [ -1.,  3.],
       [ -1.,  3.]])
```

Python automatically broadcasts arrays in this manner. Sometimes, however, how we should broadcast is ambiguous. In these cases, we can use the `newaxis` constant to specify how we want to broadcast:

```

>>> a = np.zeros((2,2), float)
>>> b = np.array([-1., 3.], float)
>>> a
array([[ 0.,  0.],
       [ 0.,  0.]])
>>> b
array([-1.,  3.])
>>> a + b
array([[ 0.,  3.],
       [ 0.,  3.]])
>>> a + b[np.newaxis,:]
array([[ 0.,  3.],
       [ 0.,  3.]])
>>> a + b[:,np.newaxis]
array([[ 0., -1.],
       [ 0.,  3.]])

```

In addition to the standard operators, NumPy offers a large library of common mathematical functions that can be applied elementwise to arrays. Among these are the functions: `abs`, `sign`, `sqrt`, `log`, `log10`, `exp`, `sin`, `cos`, `tan`, `arcsin`, `arccos`, `arctan`, `sinh`, `cosh`, `tanh`, `arcsinh`, `arccosh`, and `arctanh`.

```

>>> a = np.array([1, 4, 9], float)
>>> np.sqrt(a)
array([ 1.,  2.,  3.])

```

The functions `floor`, `ceil`, and `rint` give the lower, upper, or nearest (rounded) integer:

```

>>> a = np.array([1.1, 1.5, 1.9], float)
>>> np.floor(a)
array([ 1.,  1.,  1.])
>>> np.ceil(a)
array([ 2.,  2.,  2.])
>>> np.rint(a)
array([ 1.,  2.,  2.])

```

Also included in the NumPy module are two important mathematical constants:

```

>>> np.pi
3.1415926535897931
>>> np.e
2.7182818284590451

```

Array iteration

It is possible to iterate over arrays in a manner similar to that of lists:

```

>>> a = np.array([1, 4, 5], int)
>>> for x in a:
...     print(x)
... <hit return>
1
4
5

```


For multidimensional arrays, iteration proceeds over the first axis such that each loop returns a subsection of the array:

```
>>> a = np.array([[1, 2], [3, 4], [5, 6]], float)
>>> for x in a:
...     print(x)
... <hit return>
[ 1.  2.]
[ 3.  4.]
[ 5.  6.]
```

Multiple assignment can also be used with array iteration:

```
>>> a = np.array([[1, 2], [3, 4], [5, 6]], float)
>>> for (x, y) in a:
...     print(x * y)
... <hit return>
2.0
12.0
30.0
```

Basic array operations

Many functions exist for extracting whole-array properties. The items in an array can be summed or multiplied:

```
>>> a = np.array([2, 4, 3], float)
>>> a.sum()
9.0
>>> a.prod()
24.0
```

We can also apply these operations to array slices:

```
>>> a[1:].sum()
7.0
```

In this example, member functions of the arrays were used. Alternatively, standalone functions in the NumPy module can be accessed:

```
>>> np.sum(a)
9.0
>>> np.prod(a)
24.0
```

For most of the routines described below, both standalone and member functions are available.

A number of routines enable computation of statistical quantities in array datasets, such as the mean (average), variance, and standard deviation:

```
>>> a = np.array([2, 1, 9], float)
>>> a.mean()
4.0
>>> a.var()
```

```
12.666666666666666
>>> a.std()
3.5590260840104371
```

It's also possible to find the minimum and maximum element values:

```
>>> a = np.array([2, 1, 9], float)
>>> a.min()
1.0
>>> a.max()
9.0
```

The `argmin` and `argmax` functions return the array indices of the minimum and maximum values:

```
>>> a = np.array([2, 1, 9], float)
>>> a.argmin()
1
>>> a.argmax()
2
```

For multidimensional arrays, each of the functions thus far described can take an optional argument `axis` that will perform an operation along only the specified axis, placing the results in a return array:

```
>>> a = np.array([[0, 2], [3, -1], [3, 5]], float)
>>> a.mean(axis=0)
array([ 2.,  2.]) >>>
a.mean(axis=1)
array([ 1.,  1.,
 4.]) >>>
a.min(axis=1) array([
0., -1.,  3.]) >>>
a.max(axis=0) array([
3.,  5.])
```

Like lists, arrays can be sorted:

```
>>> a = np.array([6, 2, 5, -1, 0], float)
>>> a.sort()
>>> a
array([-1.,  0.,  2.,  5.,  6.])
```

Values in an array can be "clipped" to be within a prespecified range. This is the same as applying `min(max(x, minval), maxval)` to each element `x` in an array.

```
>>> a = np.array([6, 2, 5, -1, 0], float)
>>> a.clip(0, 5) array([ 5.,  2.,  5.,  0.,  0.])
```

Unique elements can be extracted from an array:

```
>>> a = np.array([1, 1, 4, 5, 5, 5, 7], float)
>>> np.unique(a) array([ 1.,  4.,  5.,  7.])
```

For two dimensional arrays, the diagonal can be extracted:

```
>>> a = np.array([[1, 2], [3, 4]], float)
>>> a.diagonal()
array([ 1.,  4.])
```

Comparison operators and value testing

Boolean comparisons can be used to compare members elementwise on arrays of equal size. The return value is an array of Boolean `True` / `False` values:

```
>>> a = np.array([1, 3, 0], float)
>>> b = np.array([0, 3, 2], float)
>>> a > b
array([ True, False, False], dtype=bool)
>>> a == b
array([False,  True, False], dtype=bool)
>>> a <= b array([False,  True,
True], dtype=bool)
```

The results of a Boolean comparison can be stored in an array:

```
>>> c = a > b >>> c array([ True,
False, False], dtype=bool)
```

Arrays can be compared to single values using broadcasting:

```
>>> a = np.array([1, 3, 0], float)
>>> a > 2 array([False,  True, False], dtype=bool)
```

The `any` and `all` operators can be used to determine whether or not any or all elements of a Boolean array are true:

```
>>> a = np.array([1, 3, 0], float)
>>> any(a > 2)
True
>>> all(a > 2) False
```

Compound Boolean expressions can be applied to arrays on an element-by-element basis using special functions `logical_and`, `logical_or`, and `logical_not`.

```
>>> a = np.array([1, 3, 0], float)
>>> np.logical_and(a > 0, a < 3) array([ True, False,
False], dtype=bool)
>>> b = np.array([True, False, True], bool)
>>> np.logical_not(b)
array([False,  True, False], dtype=bool) >>> c =
np.array([False, True, False], bool)
>>> np.logical_or(b, c) array([ True,  True,  True],
dtype=bool)
```

The `where` function forms a new array from two arrays of equivalent size using a Boolean filter to choose between elements of the two. Its basic syntax is `where (boolarray, truearray, falsearray)`:

```
>>> a = np.array([1, 3, 0], float)
>>> np.where(a != 0, 1 / a, a) array([ 1.          ,
0.33333333,  0.          ])
```

Broadcasting can also be used with the where function:

```
>>> np.where(a > 0, 3, 2)
array([3, 3, 2])
```

A number of functions allow testing of the values in an array. The `nonzero` function gives a tuple of indices of the nonzero values in an array. The number of items in the tuple equals the number of axes of the array:

```
>>> a = np.array([[0, 1], [3, 0]], float)
>>> a.nonzero()
(array([0, 1]), array([1, 0]))
```

It is also possible to test whether or not values are NaN ("not a number") or finite:

```
>>> a = np.array([1, np.NaN, np.Inf], float)
>>> a
array([ 1., NaN, Inf])
>>> np.isnan(a)
array([False,  True, False], dtype=bool)
>>> np.isfinite(a) array([ True, False, False], dtype=bool)
```

Although here we used NumPy constants to add the NaN and infinite values, these can result from standard mathematical operations.

Array item selection and manipulation

We have already seen that, like lists, individual elements and slices of arrays can be selected using bracket notation. Unlike lists, however, arrays also permit selection using other arrays. That is, we can use *array selectors* to filter for specific subsets of elements of other arrays.

Boolean arrays can be used as array selectors:

```
>>> a = np.array([[6, 4], [5, 9]], float)
>>> a >= 6
array([[ True, False],
       [False,  True]], dtype=bool)
>>> a[a >= 6]
array([ 6.,  9.])
```

Notice that sending the Boolean array given by `a >= 6` to the bracket selection for `a`, an array with only the `True` elements is returned. We could have also stored the selector array in a variable:

```
>>> a = np.array([[6, 4], [5, 9]], float)
>>> sel = (a >= 6)
>>> a[sel]
array([ 6.,  9.])
```

More complicated selections can be achieved using Boolean expressions:

```
>>> a[np.logical_and(a > 5, a < 9)]
>>> array([ 6.])
```

In addition to Boolean selection, it is possible to select using integer arrays. Here, the integer arrays contain the *indices* of the elements to be taken from an array. Consider the following onedimensional example:

```
>>> a = np.array([2, 4, 6, 8], float)
>>> b = np.array([0, 0, 1, 3, 2, 1], int)
>>> a[b] array([ 2.,  2.,  4.,  8.,  6.,  4.])
```

In other words, we take the 0th, 0th, 1st, 3rd, 2nd, and 1st elements of *a*, in that order, when we use *b* to select elements from *a*. Lists can also be used as selection arrays:

```
>>> a = np.array([2, 4, 6, 8], float)
>>> a[[0, 0, 1, 3, 2, 1]] array([ 2.,  2.,  4.,
8.,  6.,  4.])
```

For multidimensional arrays, we have to send multiple one-dimensional integer arrays to the selection bracket, one for each axis. Then, each of these selection arrays is traversed in sequence: the first element taken has a first axis index taken from the first member of the first selection array, a second index from the first member of the second selection array, and so on. An example:

```
>>> a = np.array([[1, 4], [9, 16]], float)
>>> b = np.array([0, 0, 1, 1, 0], int)
>>> c = np.array([0, 1, 1, 1, 1], int)
>>> a[b,c] array([ 1.,  4., 16., 16.,  4.])
```

A special function `take` is also available to perform selection with integer arrays. This works in an identical manner as bracket selection:

```
>>> a = np.array([2, 4, 6, 8], float)
>>> b = np.array([0, 0, 1, 3, 2, 1], int)
>>> a.take(b)
array([ 2.,  2.,  4.,  8.,  6.,  4.])
```

`take` also provides an `axis` argument, such that subsections of a multi-dimensional array can be taken across a given dimension.

```
>>> a = np.array([[0, 1], [2, 3]], float)
>>> b = np.array([0, 0, 1], int)
>>> a.take(b,
axis=0) array([[ 0.,
1.],          [ 0.,
1.],
[ 2.,  3.]]) >>>
a.take(b, axis=1)
array([[ 0.,  0.,  1.],
[ 2.,  2.,  3.]])
```

The opposite of the `take` function is the `put` function, which will take values from a source array and place them at specified indices in the array calling `put`.

```
>>> a = np.array([0, 1, 2, 3, 4, 5], float)
>>> b = np.array([9, 8, 7], float)
>>> a.put([0, 3], b)
>>> a array([ 9.,  1.,  2.,  8.,  4.,  5.])
```

Note that the value 7 from the source array *b* is not used, since only two indices [0, 3] are specified. The source array will be repeated as necessary if not the same size:

```
>>> a = np.array([0, 1, 2, 3, 4, 5], float)
```

```
>>> a.put([0, 3], 5)
>>> a array([ 5.,  1.,  2.,  5.,  4.,  5.]
```

Vector and matrix mathematics

NumPy provides many functions for performing standard vector and matrix multiplication routines. To perform a dot product,

```
>>> a = np.array([1, 2, 3], float)
>>> b = np.array([0, 1, 1], float)
>>> np.dot(a, b)
5.0
```

The `dot` function also generalizes to matrix multiplication:

```
>>> a = np.array([[0, 1], [2, 3]], float)
>>> b = np.array([2, 3], float) >>> c =
np.array([[1, 1], [4, 0]], float)
>>> a
array([[ 0.,  1.],
       [ 2.,  3.]]) >>>
np.dot(b, a) array([
  6., 11.]) >>>
np.dot(a, b) array([
  3., 13.]) >>>
np.dot(a, c)
array([[ 4.,  0.],
       [14.,  2.]]) >>>
np.dot(c, a)
array([[ 2.,  4.],
       [ 0.,  4.]])
```

It is also possible to generate inner, outer, and cross products of matrices and vectors. For vectors, note that the inner product is equivalent to the dot product:

```
>>> a = np.array([1, 4, 0], float)
>>> b = np.array([2, 2, 1], float)
>>> np.outer(a, b)
array([[ 2.,  2.,
  1.],      [ 8.,
  8.,  4.],
        [ 0.,  0.,
  0.]])
>>> np.inner(a, b)
10.0
>>> np.cross(a, b)
array([ 4., -1., -
  6.]])
```

NumPy also comes with a number of built-in routines for linear algebra calculations. These can be found in the sub-module `linalg`. Among these are routines for dealing with matrices and their inverses. The determinant of a matrix can be found:

```
>>> a = np.array([[4, 2, 0], [9, 3, 7], [1, 2, 1]], float)
>>> a
array([[ 4.,  2.,  0.],
       [ 9.,  3.,  7.],
       [ 1.,  2.,  1.]])
>>> np.linalg.det(a)
-53.999999999999993
```

One can find the eigenvalues and eigenvectors of a matrix:

```
>>> vals, vecs = np.linalg.eig(a)
>>> vals
array([ 9.          ,  2.44948974, -2.44948974])
>>> vecs
array([[ -0.3538921 , -0.56786837,  0.27843404],          [-0.88473024,
  0.44024287, -0.89787873],
       [ -0.30333608,  0.69549388,  0.34101066]])
```

The inverse of a matrix can be found:

```
>>> b = np.linalg.inv(a)
>>> b
array([[ 0.14814815,  0.07407407, -0.25925926],
       [ 0.2037037 , -0.14814815,  0.51851852],
       [-0.27777778,  0.11111111,  0.11111111]])
>>> np.dot(a, b)
array([[ 1.00000000e+00,  5.55111512e-17,  2.22044605e-16],
       [ 0.00000000e+00,  1.00000000e+00,  5.55111512e-16],
       [ 1.11022302e-16,  0.00000000e+00,  1.00000000e+00]])
```

Singular value decomposition (analogous to diagonalization of a nonsquare matrix) can also be performed:

```
>>> a = np.array([[1, 3, 4], [5, 2, 3]], float)
>>> U, s, Vh = np.linalg.svd(a)
>>> U
array([[ -0.6113829 , -0.79133492],          [-0.79133492,  0.6113829 ]])
>>> s
array([ 7.46791327,  2.86884495])
>>> Vh
array([[ -0.61169129, -0.45753324, -
  0.64536587],          [ 0.78971838, -0.40129005,
 -0.46401635],
       [ -0.046676 , -0.79349205,
  0.60678804]])
```

Polynomial mathematics

NumPy supplies methods for working with polynomials. Given a set of roots, it is possible to show the polynomial coefficients:

```
>>> np.poly([-1, 1, 1, 10])
array([ 1, -11,  9,  11, -10])
```

Here, the return array gives the coefficients corresponding to $x^4 - 11x^3 + 9x^2 + 11x - 10$.

The opposite operation can be performed: given a set of coefficients, the root function returns all of the polynomial roots:

```
>>> np.roots([1, 4, -2, 3])
array([-4.57974010+0.j, 0.28987005+0.75566815j,
        0.28987005-0.75566815j])
```

Notice here that two of the roots of $x^3 + 4x^2 - 2x + 3$ are imaginary.

Coefficient arrays of polynomials can be integrated. Consider integrating $x^3 + x^2 + x + 1$ to $x^4/4 + x^3/3 + x^2/2 + x + C$. By default, the constant C is set to zero:

```
>>> np.polyint([1, 1, 1, 1]) array([ 0.25, 0.33333333, 0.5,
        1., 0.])
```

Similarly, derivatives can be taken:

```
>>> np.polyder([1./4., 1./3., 1./2., 1., 0.])
array([ 1., 1., 1., 1.])
```

The functions `polyadd`, `polysub`, `polymul`, and `polydiv` also handle proper addition, subtraction, multiplication, and division of polynomial coefficients, respectively.

The function `polyval` evaluates a polynomial at a particular point. Consider $x^3 - 2x^2 + 2$ evaluated at $x = 4$:

```
>>> np.polyval([1, -2, 0, 2], 4)
34
```

Finally, the `polyfit` function can be used to fit a polynomial of specified order to a set of data using a least-squares approach:

```
>>> x = [1, 2, 3, 4, 5, 6, 7, 8]
>>> y = [0, 2, 1, 3, 7, 10, 11, 19]
>>> np.polyfit(x, y, 2) array([ 0.375,
        -0.88690476, 1.05357143])
```

The return value is a set of polynomial coefficients. More sophisticated regression and interpolation routines can be found in the SciPy package.

Statistics

In addition to the `mean`, `var`, and `std` functions, NumPy supplies several other methods for returning statistical features of arrays.

The median is given by:

```
>>> a = np.array([1, 4, 3, 8, 9, 2, 3], float)
>>> np.median(a)
```



```
3.0
```

The correlation coefficient for multiple variables observed at multiple instances can be found for arrays of the form `[[x1, x2, ...], [y1, y2, ...], [z1, z2, ...], ...]` where `x`, `y`, `z` are different observables and the numbers indicate the observation times:

```
>>> a = np.array([[1, 2, 1, 3], [5, 3, 1, 8]], float)
>>> c = np.corrcoef(a)
>>> c
array([[ 1.          ,  0.72870505],
       [ 0.72870505,  1.          ]])
```

Here the return array `c[i, j]` gives the correlation coefficient for the `i`th and `j`th observables.

Similarly, the covariance for data can be found:

```
>>> np.cov(a)
array([[ 0.91666667,  2.08333333],
       [ 2.08333333,  8.91666667]])
```

Random numbers

An important part of any simulation is the ability to draw random numbers. For this purpose, we use NumPy's built-in pseudorandom number generator routines in the sub-module `random`. The numbers are *pseudo* random in the sense that they are generated deterministically from a *seed* number, but are distributed in what has statistical similarities to random fashion. NumPy uses a particular algorithm called the Mersenne Twister to generate pseudorandom numbers.

The random number seed can be set:

```
>>> np.random.seed(293423)
```

The seed is an integer value. Any program that starts with the same seed will generate exactly the same sequence of random numbers each time it is run. This can be useful for debugging purposes, but one does not need to specify the seed and in fact, when we perform multiple runs of the same simulation to be averaged together, we want each such trial to have a different sequence of random numbers. If this command is not run, NumPy automatically selects a random seed (based on the time) that is different every time a program is run.

We can generate an array of random numbers in the half-open interval `[0.0, 1.0)`:

```
>>> np.random.rand(5)
array([ 0.40783762,  0.7550402 ,  0.00919317,
        0.01713451,  0.95299583])
```

We can use the `rand` function to generate two-dimensional random arrays, or use the `resize` function:

```
>>> np.random.rand(2,3)
array([[ 0.50431753,  0.48272463,
        0.45811345],
       [ 0.18209476,  0.48631022,
        0.49590404]])
```

```
>>> np.random.rand(6).reshape((2,3)) array([[
0.72915152, 0.59423848, 0.25644881],
[ 0.75965311, 0.52151819, 0.60084796]])
```

To generate a single random number in [0.0, 1.0),

```
>>> np.random.random()
0.70110427435769551
```

To generate random integers in the range [min, max) use `randint(min, max)`:

```
>>> np.random.randint(5, 10)
9
```

NumPy Creating Arrays

Create a NumPy ndarray Object

NumPy is used to work with arrays. The array object in NumPy is called **ndarray**.

We can create a NumPy **ndarray** object by using the **array()** function.

Example

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

type(): This built-in Python function tells us the type of the object passed to it. Like in above code it shows that **arr** is **numpy.ndarray** type.

To create an **ndarray**, we can pass a list, tuple or any array-like object into the **array()** method, and it will be converted into an **ndarray**:

Example

Use a tuple to create a NumPy array:

```
import numpy as np  
  
arr = np.array((1, 2, 3, 4, 5))  
  
print(arr)
```

Dimensions in Arrays

A dimension in arrays is one level of array depth (nested arrays).

nested array: are arrays that have arrays as their elements.

0-D Arrays

0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

Example

Create a 0-D array with value 42

```
import numpy as np  
  
arr = np.array(42)  
  
print(arr)
```

1-D Arrays

An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array.

These are the most common and basic arrays.

Example

Create a 1-D array containing the values 1,2,3,4,5:

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

2-D Arrays

An array that has 1-D arrays as its elements is called a 2-D array.

These are often used to represent matrix or 2nd order tensors.

NumPy has a whole sub module dedicated towards matrix operations called `numpy.mat`

Example

Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)
```

3-D arrays

An array that has 2-D arrays (matrices) as its elements is called 3-D array.

These are often used to represent a 3rd order tensor.

Example

Create a 3-D array with two 2-D arrays, both containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(arr)
```

Check Number of Dimensions?

NumPy Arrays provides the `ndim` attribute that returns an integer that tells us how many dimensions the array have.

Example

Check how many dimensions the arrays have:

```
import numpy as np

a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

Higher Dimensional Arrays

An array can have any number of dimensions.

When the array is created, you can define the number of dimensions by using the `ndmin` argument.

Example

Create an array with 5 dimensions and verify that it has 5 dimensions:

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('number of dimensions :', arr.ndim)
```

In this array the innermost dimension (5th dim) has 4 elements, the 4th dim has 1 element that is the vector, the 3rd dim has 1 element that is the matrix with the vector, the 2nd dim has 1 element that is 3D array and 1st dim has 1 element that is a 4D array.

NumPy Array Indexing

Access Array Elements

Array indexing is the same as accessing an array element.

You can access an array element by referring to its index number.

The indexes in NumPy arrays start with 0, meaning that the first element has index 0, and the second has index 1 etc.

Example

Get the first element from the following array:

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[0])
```

Example

Get the second element from the following array.

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[1])
```

Example

Get third and fourth elements from the following array and add them.

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[2] + arr[3])
```

Access 2-D Arrays

To access elements from 2-D arrays we can use comma separated integers representing the dimension and the index of the element.

Example

Access the 2nd element on 1st dim:

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('2nd element on 1st dim: ', arr[0, 1])
```

Example

Access the 5th element on 2nd dim:

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('5th element on 2nd dim: ', arr[1, 4])
```

Access 3-D Arrays

To access elements from 3-D arrays we can use comma separated integers representing the dimensions and the index of the element.

Example

Access the third element of the second array of the first array:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

print(arr[0, 1, 2])
```

Example Explained

`arr[0, 1, 2]` prints the value 6.

And this is why:

The first number represents the first dimension, which contains two arrays:

`[[1, 2, 3], [4, 5, 6]]`

and:

`[[7, 8, 9], [10, 11, 12]]`

Since we selected 0, we are left with the first array:

`[[1, 2, 3], [4, 5, 6]]`

The second number represents the second dimension, which also contains two arrays:

[1, 2, 3]

and:

[4, 5, 6]

Since we selected **1**, we are left with the second array:

[4, 5, 6]

The third number represents the third dimension, which contains three values:

4

5

6

Since we selected **2**, we end up with the third value:

6

Negative Indexing

Use negative indexing to access an array from the end.

Example

Print the last element from the 2nd dim:

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('Last element from 2nd dim: ', arr[1, -1])
```

NumPy Array Slicing

Slicing arrays

Slicing in python means taking elements from one given index to another given index.

We pass slice instead of index like this: **[start:end]**.

We can also define the step, like this: **[start:end:step]**.

If we don't pass start its considered 0

If we don't pass end its considered length of array in that dimension

If we don't pass step its considered 1

Example

Slice elements from index 1 to index 5 from the following array:

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
  
print(arr[1:5])
```

Note: The result *includes* the start index, but *excludes* the end index.

Example

Slice elements from index 4 to the end of the array:

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
  
print(arr[4:])
```

Example

Slice elements from the beginning to index 4 (not included):

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
  
print(arr[:4])
```

Negative Slicing

Use the minus operator to refer to an index from the end:

Example

Slice from the index 3 from the end to index 1 from the end:

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
  
print(arr[-3:-1])
```

STEP

Use the **step** value to determine the step of the slicing:

Example

Return every other element from index 1 to index 5:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5:2])
```

Example

Return every other element from the entire array:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[:2])
```

Slicing 2-D Arrays

Example

From the second element, slice elements from index 1 to index 4 (not included):

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[1, 1:4])
```

Note: Remember that *second element* has index 1.

Example

From both elements, return index 2:

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[0:2, 2])
```

Example

From both elements, slice index 1 to index 4 (not included), this will return a 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[0:2, 1:4])
```

NumPy Data Types

Data Types in Python

By default Python have these data types:

- **strings** - used to represent text data, the text is given under quote marks. eg. "ABCD"
 - **integer** - used to represent integer numbers. eg. -1, -2, -3
 - **float** - used to represent real numbers. eg. 1.2, 42.42
 - **boolean** - used to represent True or False.
 - **complex** - used to represent a number in complex plain. eg. 1.0 + 2.0j, 1.5 + 2.5j
-

Data Types in NumPy

NumPy has some extra data types, and refer to data types with one character, like **i** for integers, **u** for unsigned integers etc.

Below is a list of all data types in NumPy and the characters used to represent them.

- **i** - integer
- **b** - boolean
- **u** - unsigned integer
- **f** - float
- **c** - complex float

- **m** - timedelta
 - **M** - datetime
 - **O** - object
 - **S** - string
 - **U** - unicode string
 - **V** - fixed chunk of memory for other type (void)
-

Checking the Data Type of an Array

The NumPy array object has a property called **dtype** that returns the data type of the array:

Example

Get the data type of an array object:

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr.dtype)
```

Example

Get the data type of an array containing strings:

```
import numpy as np

arr = np.array(['apple', 'banana', 'cherry'])

print(arr.dtype)
```

Creating Arrays With a Defined Data Type

We use the **array()** function to create arrays, this function can take an optional argument: **dtype** that allows us to define the expected data type of the array elements:

Example

Create an array with data type string:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4], dtype='S')  
  
print(arr)  
print(arr.dtype)
```

For `i`, `u`, `f`, `S` and `U` we can define size as well.

Example

Create an array with data type 4 bytes integer:

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4], dtype='i4')  
  
print(arr)  
print(arr.dtype)
```

What if a Value Can Not Be Converted?

If a type is given in which elements can't be casted then NumPy will raise a `ValueError`.

ValueError: In Python `ValueError` is raised when the type of passed argument to a function is unexpected/incorrect.

Example

A non integer string like 'a' can not be converted to integer (will raise an error):

```
import numpy as np  
  
arr = np.array(['a', '2', '3'], dtype='i')
```

Converting Data Type on Existing Arrays

The best way to change the data type of an existing array, is to make a copy of the array with the `astype()` method.

The `astype()` function creates a copy of the array, and allows you to specify the data type as a parameter.

The data type can be specified using a string, like `'f'` for float, `'i'` for integer etc. or you can use the data type directly like `float` for float and `int` for integer.

Example

Change data type from float to integer by using 'i' as parameter value:

```
import numpy as np

arr = np.array([1.1, 2.1, 3.1])

newarr = arr.astype('i')

print(newarr)
print(newarr.dtype)
```

Example

Change data type from float to integer by using int as parameter value:

```
import numpy as np

arr = np.array([1.1, 2.1, 3.1])

newarr = arr.astype(int)

print(newarr)
print(newarr.dtype)
```

Example

Change data type from integer to boolean:

```
import numpy as np

arr = np.array([1, 0, 3])

newarr = arr.astype(bool)

print(newarr)
print(newarr.dtype)
```

NumPy Array Copy vs View

The Difference Between Copy and View

The main difference between a copy and a view of an array is that the copy is a new array, and the view is just a view of the original array.

The copy *owns* the data and any changes made to the copy will not affect original array, and any changes made to the original array will not affect the copy.

The view *does not own* the data and any changes made to the view will affect the original array, and any changes made to the original array will affect the view.

COPY:

Example

Make a copy, change the original array, and display both arrays:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42

print(arr)
print(x)
```

The copy SHOULD NOT be affected by the changes made to the original array.

VIEW:

Example

Make a view, change the original array, and display both arrays:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42

print(arr)
print(x)
```

The view SHOULD be affected by the changes made to the original array.

Make Changes in the VIEW:

Example

Make a view, change the view, and display both arrays:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
x[0] = 31

print(arr)
print(x)
```

The original array SHOULD be affected by the changes made to the view.

Check if Array Owns it's Data

As mentioned above, copies *owns* the data, and views *does not own* the data, but how can we check this?

Every NumPy array has the attribute `base` that returns `None` if the array owns the data.

Otherwise, the `base` attribute refers to the original object.

Example

Print the value of the base attribute to check if an array owns it's data or not:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

x = arr.copy()
y = arr.view()

print(x.base)
print(y.base)
```

The copy returns `None`.
The view returns the original array.

Shape of an Array

The shape of an array is the number of elements in each dimension.

Get the Shape of an Array

NumPy arrays have an attribute called `shape` that returns a tuple with each index having the number of corresponding elements.

Example

Print the shape of a 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

print(arr.shape)
```

The example above returns `(2, 4)`, which means that the array has 2 dimensions, and each dimension has 4 elements.

Example

Create an array with 5 dimensions using `ndmin` using a vector with values 1,2,3,4 and verify that last dimension has value 4:

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('shape of array :', arr.shape)
```

What does the shape tuple represent?

Integers at every index tells about the number of elements the corresponding dimension has.

In the example above at index-4 we have value 4, so we can say that 5th (4 + 1 th) dimension has 4 elements.

NumPy Array Reshaping

[< Previous](#) [Next >](#)

Reshaping arrays

Reshaping means changing the shape of an array.

The shape of an array is the number of elements in each dimension.

By reshaping we can add or remove dimensions or change number of elements in each dimension.

Reshape From 1-D to 2-D

Example

Convert the following 1-D array with 12 elements into a 2-D array.

The outermost dimension will have 4 arrays, each with 3 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])

newarr = arr.reshape(4, 3)

print(newarr)
```

Reshape From 1-D to 3-D

Example

Convert the following 1-D array with 12 elements into a 3-D array.

The outermost dimension will have 2 arrays that contains 3 arrays, each with 2 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])

newarr = arr.reshape(2, 3, 2)

print(newarr)
```

Can We Reshape Into any Shape?

Yes, as long as the elements required for reshaping are equal in both shapes.

We can reshape an 8 elements 1D array into 4 elements in 2 rows 2D array but we cannot reshape it into a 3 elements 3 rows 2D array as that would require $3 \times 3 = 9$ elements.

Example

Try converting 1D array with 8 elements to a 2D array with 3 elements in each dimension (will raise an error):

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

newarr = arr.reshape(3, 3)

print(newarr)
```

Returns Copy or View?

Example

Check if the returned array is a copy or a view:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

print(arr.reshape(2, 4).base)
```

The example above returns the original array, so it is a view.

Unknown Dimension

You are allowed to have one "unknown" dimension.

Meaning that you do not have to specify an exact number for one of the dimensions in the reshape method.

Pass `-1` as the value, and NumPy will calculate this number for you.

Example

Convert 1D array with 8 elements to 3D array with 2x2 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

newarr = arr.reshape(2, 2, -1)

print(newarr)
```

Note: We can not pass `-1` to more than one dimension.

Flattening the arrays

Flattening array means converting a multidimensional array into a 1D array.

We can use `reshape(-1)` to do this.

Example

Convert the array into a 1D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

newarr = arr.reshape(-1)

print(newarr)
```

Note: There are a lot of functions for changing the shapes of arrays in numpy `flatten`, `ravel` and also for rearranging the elements `rot90`, `flip`, `flipplr`, `flipud` etc. These fall under Intermediate to Advanced section of numpy.

NumPy Array Iterating

Iterating Arrays

Iterating means going through elements one by one.

As we deal with multi-dimensional arrays in numpy, we can do this using basic `for` loop of python.

If we iterate on a 1-D array it will go through each element one by one.

Example

Iterate on the elements of the following 1-D array:

```
import numpy as np

arr = np.array([1, 2, 3])

for x in arr:
    print(x)
```

Iterating 2-D Arrays

In a 2-D array it will go through all the rows.

Example

Iterate on the elements of the following 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
    print(x)
```

If we iterate on a n -D array it will go through $n-1$ th dimension one by one.

To return the actual values, the scalars, we have to iterate the arrays in each dimension.

Example

Iterate on each scalar element of the 2-D array:

```
import numpy as np
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
for x in arr:  
    for y in x:  
        print(y)
```

Iterating 3-D Arrays

In a 3-D array it will go through all the 2-D arrays.

Example

Iterate on the elements of the following 3-D array:

```
import numpy as np  
  
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])  
  
for x in arr:  
    print(x)
```

To return the actual values, the scalars, we have to iterate the arrays in each dimension.

Example

Iterate down to the scalars:

```
import numpy as np  
  
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])  
  
for x in arr:  
    for y in x:  
        for z in y:  
            print(z)
```

Iterating Arrays Using `nditer()`

The function `nditer()` is a helping function that can be used from very basic to very advanced iterations. It solves some basic issues which we face in iteration, let's go through it with examples.

Iterating on Each Scalar Element

In basic `for` loops, iterating through each scalar of an array we need to use n `for` loops which can be difficult to write for arrays with very high dimensionality.

Example

Iterate through the following 3-D array:

```
import numpy as np

arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

for x in np.nditer(arr):
    print(x)
```

Iterating Array With Different Data Types

We can use `op_dtypes` argument and pass it the expected datatype to change the datatype of elements while iterating.

NumPy does not change the data type of the element in-place (where the element is in array) so it needs some other space to perform this action, that extra space is called buffer, and in order to enable it in `nditer()` we pass `flags=['buffered']`.

Example

Iterate through the array as a string:

```
import numpy as np

arr = np.array([1, 2, 3])

for x in np.nditer(arr, flags=['buffered'], op_dtypes=['S']):
    print(x)
```

Iterating With Different Step Size

We can use filtering and followed by iteration.

Example

Iterate through every scalar element of the 2D array skipping 1 element:

```
import numpy as np
```

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])  
  
for x in np.nditer(arr[:, ::2]):  
    print(x)
```

Enumerated Iteration Using ndenumerate()

Enumeration means mentioning sequence number of somethings one by one.

Sometimes we require corresponding index of the element while iterating, the `ndenumerate()` method can be used for those usecases.

Example

Enumerate on following 1D arrays elements:

```
import numpy as np  
  
arr = np.array([1, 2, 3])  
  
for idx, x in np.ndenumerate(arr):  
    print(idx, x)
```

Example

Enumerate on following 2D array's elements:

```
import numpy as np  
  
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])  
  
for idx, x in np.ndenumerate(arr):  
    print(idx, x)
```

NumPy Joining Array

Joining NumPy Arrays

Joining means putting contents of two or more arrays in a single array.

In SQL we join tables based on a key, whereas in NumPy we join arrays by axes.

We pass a sequence of arrays that we want to join to the `concatenate()` function, along with the axis. If axis is not explicitly passed, it is taken as 0.

Example

Join two arrays

```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.concatenate((arr1, arr2))

print(arr)
```

Example

Join two 2-D arrays along rows (axis=1):

```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])
arr = np.concatenate((arr1, arr2), axis=1)

print(arr)
```

Joining Arrays Using Stack Functions

Stacking is same as concatenation, the only difference is that stacking is done along a new axis.

We can concatenate two 1-D arrays along the second axis which would result in putting them one over the other, ie. stacking.

We pass a sequence of arrays that we want to join to the `concatenate()` method along with the axis. If axis is not explicitly passed it is taken as 0.

Example

```
import numpy as np

arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([4, 5, 6])  
arr = np.stack((arr1, arr2), axis=1)  
print(arr)
```

Stacking Along Rows

NumPy provides a helper function: `hstack()` to stack along rows.

Example

```
import numpy as np  
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
arr = np.hstack((arr1, arr2))  
print(arr)
```

Stacking Along Columns

NumPy provides a helper function: `vstack()` to stack along columns.

Example

```
import numpy as np  
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
arr = np.vstack((arr1, arr2))  
print(arr)
```

Stacking Along Height (depth)

NumPy provides a helper function: `dstack()` to stack along height, which is the same as depth.

Example

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.dstack((arr1, arr2))

print(arr)
```

NumPy Splitting Array

Splitting NumPy Arrays

Splitting is reverse operation of Joining.

Joining merges multiple arrays into one and Splitting breaks one array into multiple.

We use `array_split()` for splitting arrays, we pass it the array we want to split and the number of splits.

Example

Split the array in 3 parts:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

newarr = np.array_split(arr, 3)

print(newarr)
```

Note: The return value is an array containing three arrays.

If the array has less elements than required, it will adjust from the end accordingly.

Example

Split the array in 4 parts:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])
```

```
newarr = np.array_split(arr, 4)

print(newarr)
```

Note: We also have the method `split()` available but it will not adjust the elements when elements are less in source array for splitting like in example above, `array_split()` worked properly but `split()` would fail.

Split Into Arrays

The return value of the `array_split()` method is an array containing each of the split as an array.

If you split an array into 3 arrays, you can access them from the result just like any array element:

Example

Access the splitted arrays:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

newarr = np.array_split(arr, 3)

print(newarr[0])
print(newarr[1])
print(newarr[2])
```

Splitting 2-D Arrays

Use the same syntax when splitting 2-D arrays.

Use the `array_split()` method, pass in the array you want to split and the number of splits you want to do.

Example

Split the 2-D array into three 2-D arrays.

```
import numpy as np

arr = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
```

```
newarr = np.array_split(arr, 3)

print(newarr)
```

The example above returns three 2-D arrays.

Let's look at another example, this time each element in the 2-D arrays contains 3 elements.

Example

Split the 2-D array into three 2-D arrays.

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15],
               [16, 17, 18]])

newarr = np.array_split(arr, 3)

print(newarr)
```

The example above returns three 2-D arrays.

In addition, you can specify which axis you want to do the split around.

The example below also returns three 2-D arrays, but they are split along the row (axis=1).

Example

Split the 2-D array into three 2-D arrays along rows.

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15],
               [16, 17, 18]])

newarr = np.array_split(arr, 3, axis=1)

print(newarr)
```

An alternate solution is using `hsplit()` opposite of `hstack()`

Example

Use the `hsplit()` method to split the 2-D array into three 2-D arrays along rows.

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15],
                [16, 17, 18]])

newarr = np.hsplit(arr, 3)

print(newarr)
```